

Weather Tracker Application

1 Weather Tracker Application

🔖 Python: Project Overview

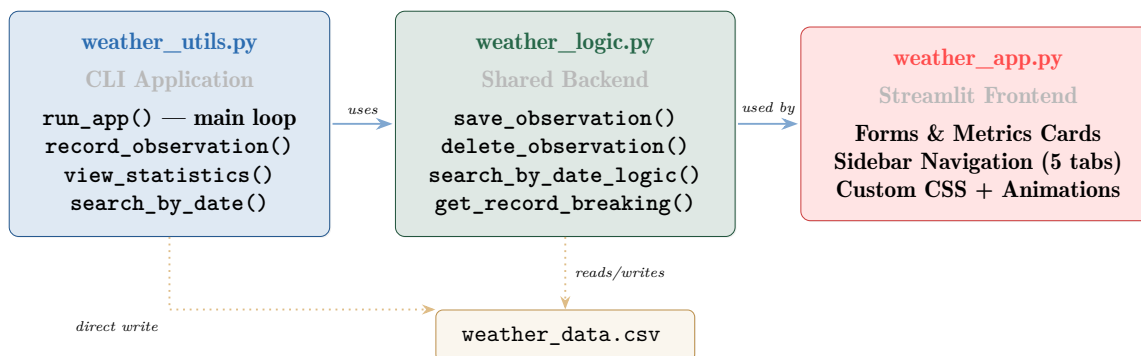
Weather enthusiasts want a simple application to track local weather observations and analyze weather patterns over time. The solution: a **three-file Python application** that can run as either a command-line tool or a modern Streamlit dashboard.

Core features:

- **Record observations** — date, temperature (C), condition, humidity (%), wind speed (km/h) with full input validation
- **View statistics** — mean/min/max temperature, monthly filters, year-over-year comparison
- **Search by date** — find all observations for a specific day
- **View all records** — browse the full dataset, delete with confirmation
- **Record-breaking detection** — automatically flags temperature outliers using the Interquartile Range (IQR) method

1.1 Three-File Architecture

The application is split across three Python files and a CSV data store, each with a clear responsibility:



File	Layer	Responsibility
<code>weather_utils.py</code>	CLI Frontend	Menu loop, user input, validation, display
<code>weather_logic.py</code>	Backend (shared)	CSV CRUD, search, IQR outlier detection
<code>weather_app.py</code>	Streamlit Frontend	Page layout, CSS, sidebar nav, forms, charts (calls <code>w1.</code> from backend)
<code>weather_data.csv</code>	Data Store	Persistent storage (5 columns of observations)

⚠ Warning: Don't Duplicate — Import Instead

In the original project, `weather_utils.py` re-implements CSV logic that already exists in `weather_logic.py` instead of importing it. **Why this is bad:**

- If you fix a bug in one file, you must remember to fix it in the other
- Any new feature must be written twice — more work, more bugs

- Inconsistent behaviour between CLI and Streamlit versions over time

The **correct approach** is to replace the duplicated logic with a single import:

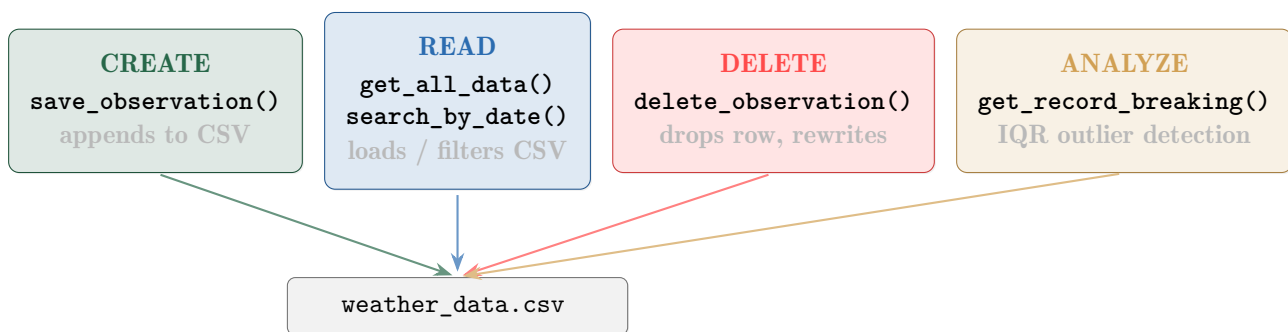
```
# Inside weather_utils.py -- add this at the top:
import weather_logic as wl

# Then replace inline CSV code with:
def record_observation():
    # ... validate input ...
    wl.save_observation(date, temp, condition, humidity, wind)

def view_statistics():
    df = wl.get_all_data()
    # ... display stats ...
```

1.2 Data Layer: Shared Backend (weather_logic.py)

The backend file handles all CSV interactions. Every function either reads or writes the data file `weather_data.csv`:



The **IQR method** for record-breaking temperatures works by:

1. Calculating Q1 (25th percentile) and Q3 (75th percentile) of all recorded temperatures
2. Computing the interquartile range: $IQR = Q3 - Q1$
3. Setting thresholds: $low = Q1 - 1.5 \times IQR$, $high = Q3 + 1.5 \times IQR$
4. Flagging any temperature below `low` or above `high` as a record-breaking outlier

Here is the full backend — only **68 lines**, reused by both frontends:

We start with imports and the CSV path constant:

```
1 import pandas as pd
2 import os
3 from datetime import datetime
4
5 CSV_FILE = 'weather_data.csv'
```

`initialize_data()` handles the logic:

```
1 def initialize_data():
2     """Creates dummy data if CSV doesn't exist."""
3     if not os.path.exists(CSV_FILE):
4         dummy_data = {
5             'Date': ['06-25-2022', '06-26-2023', '06-27-2021',
```

```

6         '06-28-2023', '06-29-2026'],
7         'Temperature': [35.0, 38.5, 34.0, 40.0, 39.0],
8         'Conditions': ['Sunny', 'Sunny', 'Cloudy', 'Sunny', '
          Rainy'],
9         'Humidity': ['40%', '45%', '60%', '35%', '70%'],
10        'Wind_Speed': [15.0, 20.0, 10.0, 25.0, 5.0]
11    }
12    pd.DataFrame(dummy_data).to_csv(CSV_FILE, index=False)

```

`get_all_data()` handles the logic:

```

1 def get_all_data():
2     if os.path.exists(CSV_FILE):
3         return pd.read_csv(CSV_FILE)
4     return pd.DataFrame()

```

`save_observation()` handles the logic:

```

1 def save_observation(date_obj, temperature, condition,
2                       humidity, wind_speed):
3     data = {
4         'Date': date_obj.strftime("%m-%d-%Y"),
5         'Temperature': float(temperature),
6         'Conditions': condition.capitalize(),
7         'Humidity': f"{humidity}%",
8         'Wind_Speed': float(wind_speed),
9     }
10    pd.DataFrame([data]).to_csv(CSV_FILE, mode='a',
11                                header=False, index=False)

```

`search_by_date_logic()` handles the logic:

```

1 def search_by_date_logic(search_date_obj):
2     df = get_all_data()
3     if df.empty:
4         return []
5     df['Date'] = pd.to_datetime(df['Date'], format='%m-%d-%Y',
6                                 errors='coerce').dt.date
7     searched = df[df['Date'] == search_date_obj]
8     return searched.to_dict('records')

```

`delete_observation()` handles the logic:

```

1 def delete_observation(index_to_remove):
2     df = get_all_data()
3     if index_to_remove in df.index:
4         df = df.drop(index=index_to_remove)
5         df.to_csv(CSV_FILE, index=False)
6         return True
7     return False

```

`get_record_breaking()` handles the logic:

```

1 def get_record_breaking():
2     """IQR-based outlier detection."""
3     df = get_all_data()
4     if df.empty:
5         return pd.DataFrame()
6     q1 = df['Temperature'].quantile(0.25)
7     q3 = df['Temperature'].quantile(0.75)
8     iqr = q3 - q1

```

```

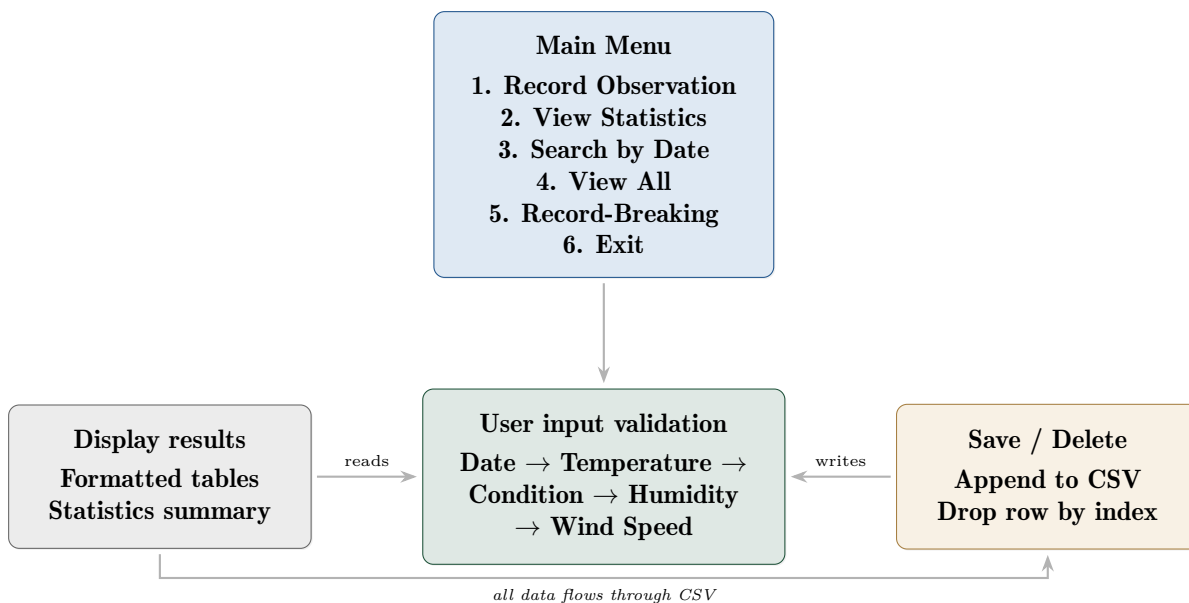
9     lowest = q1 - (iqr * 1.5)
10    highest = q3 + (iqr * 1.5)
11    outliers = df[(df['Temperature'] < lowest) |
12                  (df['Temperature'] > highest)]
13    return outliers

```

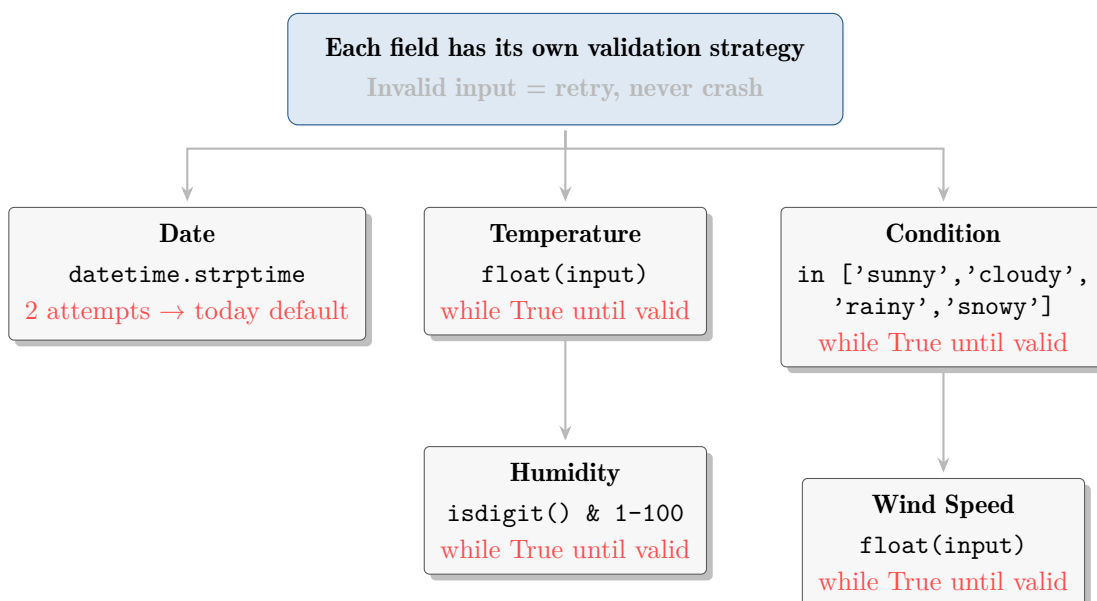
1.3 CLI Application: Input Validation in Action (weather_utils.py)

The CLI version (`weather_utils.py`) is a standalone script with its own complete menu system. Unlike `weather_app.py`, it does **not** import `weather_logic.py` — the data logic is duplicated inline (a real-world example of why modularity matters).

The core is a **menu loop** that keeps the app running until the user chooses Exit:



Each user input in `record_observation()` is validated independently with a `while True` loop that keeps asking until the user provides valid data:



Here is the core pattern — the `record_observation()` function validates every field before writing to CSV:

DATE: try/except with fallback to today:

```
1 def record_observation():
2     print("\n[ Recording a New Observation ]")
3     observation_date = input("Date (MM-DD-YYYY): ")
4     try:
5         observation_date = datetime.strptime(
6             observation_date, "%m-%d-%Y").date()
7     except ValueError:
8         observation_date = input("1 more attempt: ")
9         try:
10            observation_date = datetime.strptime(
11                observation_date, "%m-%d-%Y").date()
12        except ValueError:
13            observation_date = datetime.now().date()
14            print(f"Using today: {observation_date}")
```

TEMPERATURE: while True with try/except:

```
1     while True:
2         temperature = input('Temperature (Celsius): ')
3         try:
4             temperature = float(temperature)
5             break
6         except ValueError:
7             print("Invalid. Enter a number.")
```

CONDITION: whitelist validation:

```
1     valid = ['sunny', 'cloudy', 'rainy', 'snowy']
2     while True:
3         condition = input('Condition: ').strip().lower()
4         if condition in valid:
5             break
6         print("Must be: Sunny, Cloudy, Rainy, Snowy")
```

HUMIDITY: `isdigit()` + range check:

```
1     while True:
2         humidity = input('Humidity (%): ')
3         if humidity.isdigit() and 1 <= int(humidity) <= 100:
4             break
5         print("Enter 1-100.")
```

WIND SPEED: while True with try/except:

```
1     while True:
2         wind_speed = input('Wind speed (km/h): ')
3         try:
4             wind_speed = float(wind_speed)
5             break
6         except ValueError:
7             print("Invalid. Enter a number.")
```

SAVE: build the row dict and append to CSV:

```
1     data = {
2         'Date': observation_date.strftime("%m-%d-%Y"),
3         'Temperature': temperature,
```

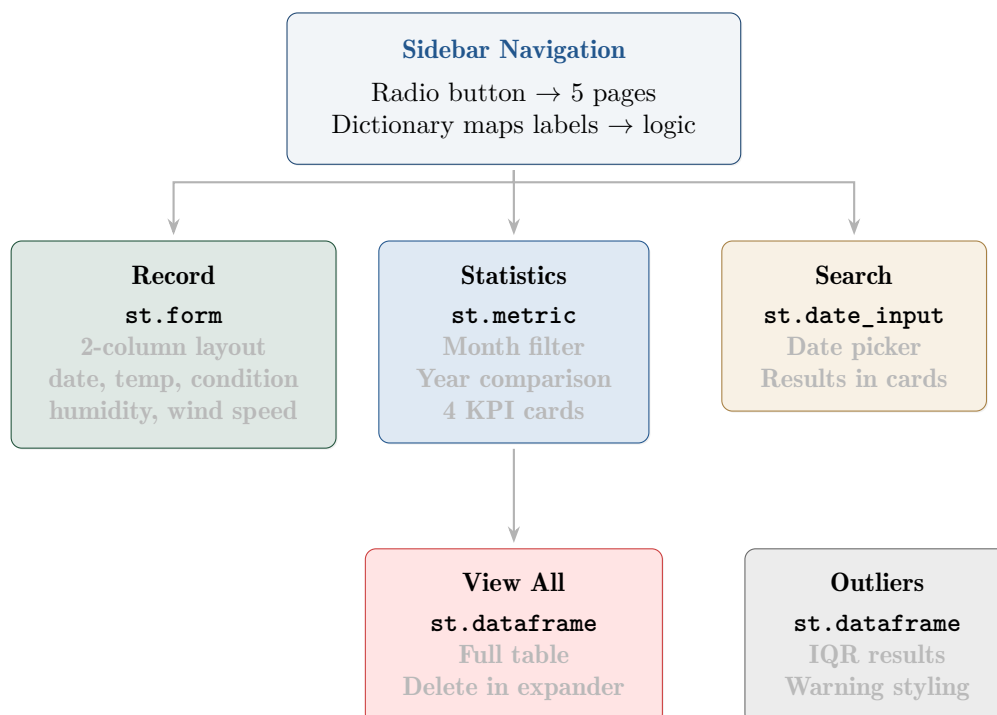
```

4      'Conditions': condition.capitalize(),
5      'Humidity': f"{humidity}%",
6      'Wind_Speed': wind_speed,
7  }
8  pd.DataFrame([data]).to_csv(CSV_FILE,
9                              mode='a', header=False, index=
10                                 False)
11  print('Observation saved!')

```

1.4 Streamlit Frontend: Interactive Dashboard (weather_app.py)

The Streamlit version provides a polished GUI with **sidebar navigation**, styled buttons, and real-time metrics. It imports `weather_logic.py` as `wl` and delegates all data work to the backend.



The app uses **custom CSS** for smooth hover animations, card-style containers, and readable sidebar text (avoiding the common "invisible text in dark mode" bug):

Custom CSS for polished UI:

```

1  st.set_page_config(page_title="Weather Tracker",
2                      layout="wide")
3
4  # Custom CSS for polished UI
5  st.markdown("""<style>
6      /* Buttons lift on hover */
7      div.stButton > button {
8          background-color: #2E86C1; color: white;
9          border-radius: 8px; border: none;
10         padding: 10px 24px; font-weight: bold;

```

Button hover effects and card transitions:

```

1      transition: all 0.4s cubic-bezier(0.25,0.8,0.25,1);
2  }

```

```

3     div.stButton > button: hover {
4         background-color: #1B4F72;
5         transform: translateY(-2px);
6         box-shadow: 0px 6px 15px rgba(0,0,0,0.2);
7     }
8     /* Cards lift on hover + blue border highlight */
9     div[data-testid="stForm"]:hover,
10    div[data-testid="stVerticalBlockBorderWrapper"]:hover {

```

Sidebar styling for dark mode compatibility:

```

1         transform: translateY(-3px);
2         box-shadow: 0px 8px 20px rgba(0,0,0,0.08);
3         border-color: #2E86C1;
4     }
5     /* Metric values styled blue */
6     div[data-testid="stMetricValue"] {
7         font-size: 2rem; color: #2E86C1;
8     }
9     /* Sidebar: fix invisible text in dark mode */
10    [data-testid="stSidebar"] { background-color: #F8F9F9; }

```

Sidebar navigation via dictionary:

```

1     [data-testid="stSidebar"] p, [data-testid="stSidebar"] span,
2     [data-testid="stSidebar"] label, [data-testid="stSidebar"] h1
3     ,
4     [data-testid="stSidebar"] h2, [data-testid="stSidebar"] h3 {
5         color: #2C3E50 !important;
6     }
7
8 </style>""", unsafe_allow_html=True)
9
10 # Sidebar navigation via dictionary
11 menu_options = {
12     "Record Observation": "Record a new observation",

```

Sidebar menu dictionary:

```

1     "Weather Statistics": "View weather statistics",
2     "Search by Date": "Search observations by date",
3     "View All Records": "View all observations",
4     "Record-Breaking": "View Record-Breaking Temperatures"
5 }
6 choice_label = st.sidebar.radio("Go to:",
7                                 list(menu_options.keys()))
8 choice = menu_options[choice_label]

```

The **record form** uses `st.form` to batch all inputs into a single rerun. The **statistics page** displays metrics with `st.metric` and supports month filtering + year-over-year comparison. The **delete function** is hidden in a `st.expander` labelled "Danger Zone" to prevent accidents:

RECORD FORM (single submit):

```

1 # --- RECORD FORM (single submit) ---
2 with st.form("record_weather_form", border=True):
3     col1, col2 = st.columns(2)
4     with col1:
5         obs_date = st.date_input("Date of Observation")
6         temp = st.number_input("Temperature (C)", value
7                                =30.0)

```

Form columns layout:

```

1         condition = st.selectbox("Condition",
2                                   ['Sunny', 'Cloudy', 'Rainy', '
3                                   Snowy'])
4     with col2:
5         humidity = st.number_input("Humidity (%)", 1, 100)
6         wind      = st.number_input("Wind Speed (km/h)",
7                                     min_value=0.0)

```

Submit button with success feedback:

```

1     if st.form_submit_button("Save Observation"):
2         wl.save_observation(obs_date, temp,
3                             condition, humidity, wind)
4         st.success("Observation saved!")

```

STATISTICS PAGE:

```

1 # --- STATISTICS PAGE ---
2 df = wl.get_all_data()
3 df['Date'] = pd.to_datetime(df['Date'], format="%m-%d-%Y")
4 df['Year'] = df['Date'].dt.year
5
6 with st.container(border=True):
7     st.markdown("#### Filter Options")

```

Filter and metric cards:

```

1     filter_month = st.selectbox("Month", ["All"] + list(range
2                                         (1,13)))
3     compare_year = st.selectbox("Compare Year",
4                                  ["None"] + sorted(df['Year'].
5                                                    unique()))
6
7 m1, m2, m3, m4 = st.columns(4)
8 m1.metric("Average Temp", f"{df['Temperature'].mean():.1f}C")
9 m2.metric("Max Temp", f"{df['Temperature'].max():.1f}C")

```

The remaining metric cards:

```

1 m3.metric("Min Temp", f"{df['Temperature'].min():.1f}C")
2 m4.metric("Most Common",
3           df['Conditions'].mode()[0] if not df['Conditions'].mode
4           ().empty else "N/A")

```

DELETE (hidden in expander):

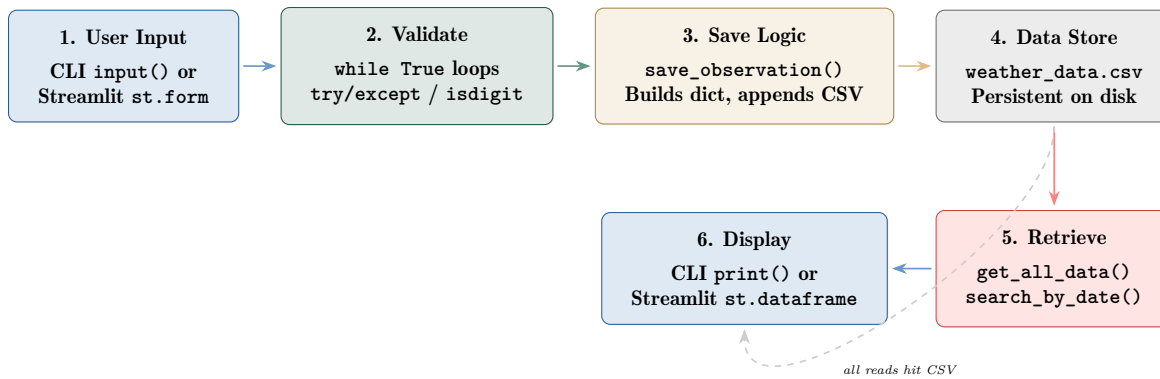
```

1 # --- DELETE (hidden in expander) ---
2 with st.expander(" Danger Zone: Remove an Observation"):
3     st.warning("Deleting a record cannot be undone.")
4     remove_index = st.number_input("Index to remove",
5                                     min_value=0, step=1)
6     if st.button("Delete Record", type="primary"):
7         if wl.delete_observation(remove_index):
8             st.success("Deleted! Refreshing...")
9             st.rerun()

```

1.5 End-to-End Data Flow

Here is how a single observation travels through the entire system:



📖 Python: Key Takeaway

The Weather Tracker project demonstrates three essential software design patterns:

1. **Separation of concerns** — frontend (CLI/Streamlit) is decoupled from backend (data logic) via a shared module
2. **Input validation first** — never trust raw user input; validate every field before saving
3. **CSV as persistence** — simple file-based storage is enough for small projects; the same pattern scales to databases later

Practice version (practice/3.6 Streamlit/) adds a 5th **Edit/Delete** tab to the Streamlit app, allowing in-place editing of observations. The `myutils.py` utility module (in practice/3.5 Python Modules and Scripting/) contains reusable functions for temperature conversion (`celsius_to_fahrenheit`, `fahrenheit_to_celsius`) and statistical analysis (`calculate_statistics`), demonstrating how to organise reusable code into modules — exactly the same pattern used by `weather_logic.py`.